INFO 7375 Systems Design

Midterm: Virtual Waiting Room

Option C: Redis / ElastiCache

Difficulty Coefficient: 1.08

27 October 2026

1. Design Overview

The Problem

The Week 5 event-driven order system has no admission control. When 1,520 visitors arrive in 25 seconds and the backend is capped at 40 requests/second, approximately 41% of fans (624 out of 1,520) are randomly rejected with HTTP 429 errors. There is no fairness, no ordering, and bots have the same chance of getting through as real fans.

The baseline stampede run shows the problem clearly:

```
indow ends   joins   polls  orders  rejected  throttled  errors  orders/s   orders accepted
-------------------------------------------------------------------------------------------
9:56:30          0       0       0         0          0       0       0.0
9:56:40          0       0      58         0          0       0       5.8   ###
9:56:50          0       0     650         0        373       0      65.0   ########################################
9:57:00          0       0     188         0        251       0      18.8   ##########

isitors: 1520   failed: 0
rders accepted: 896   rejected at gate: 0   throttled: 624   errors: 0
eak orders accepted per second (best 10 s window): 65.0
```

Figure 1: Baseline stampede -- 624 throttled, peak 65 orders/sec

The baseline dashboard confirms the damage -- a spike of 593 4xx errors at the API Gateway level:

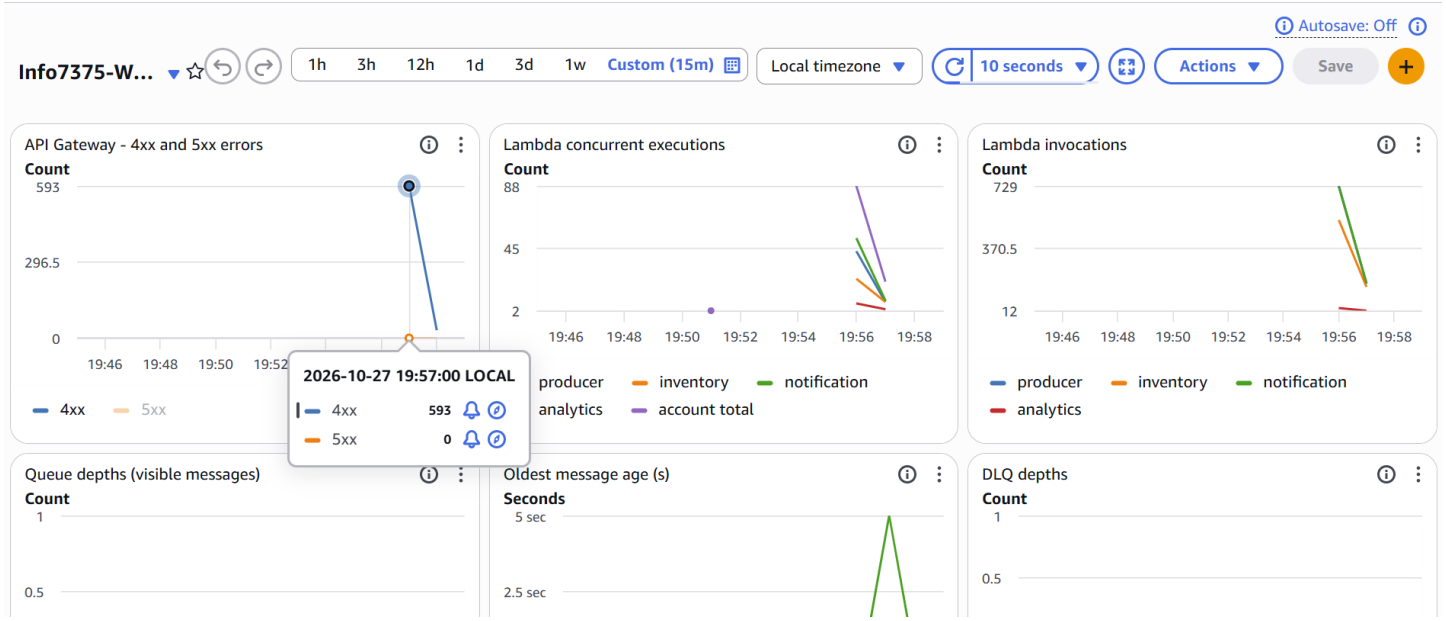


Figure 2: Baseline dashboard -- API Gateway errors, Lambda concurrency, invocations

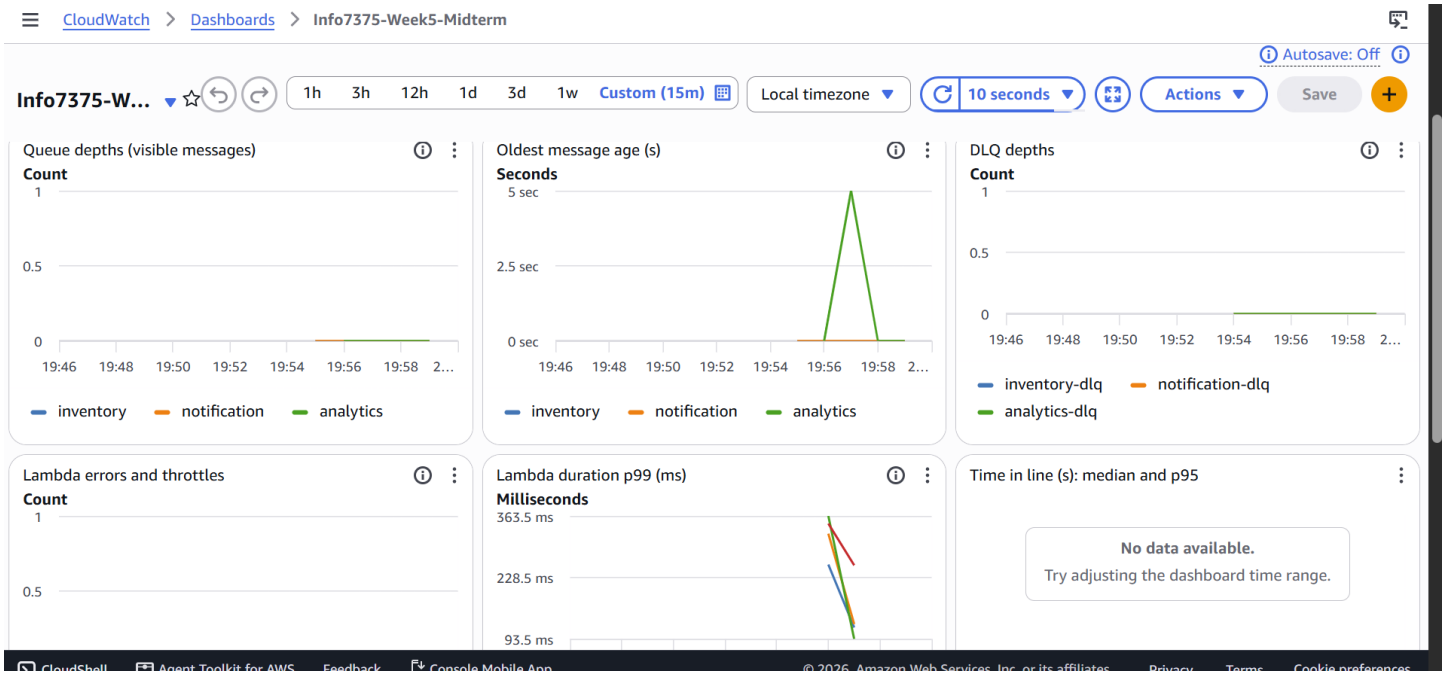


Figure 3: Baseline dashboard -- Queue depths, message age, DLQ, Lambda duration

Our Solution: A Redis-Based Virtual Waiting Room

We chose Option C (Redis/ElastiCache) to implement a virtual waiting room that sits in front of POST /orders. The waiting room uses Redis's atomic INCR command to assign each visitor a sequential queue position, then uses a time-based computation to determine when each visitor's turn has come. This is the same architectural pattern used by SeatGeek and the AWS reference architecture for virtual waiting rooms.

Why Option C (Redis) Over the Other Options

Redis was chosen for one specific technical reason: its INCR command can handle over 100,000 atomic increments per second on a single key with sub-millisecond latency. In contrast, DynamoDB's atomic increment on a single item (Option A) creates a "hot partition" -- at high concurrency, all writes to one item contend on the same physical storage partition, causing throttling. For a waiting room counter that every single visitor hits, Redis is the right tool because the workload is pure atomic coordination, not data storage.

This is also why the real-world implementations (AWS's own reference architecture and SeatGeek's "Bouncer" service) use Redis, not DynamoDB, for this exact use case.

2. Architecture

What Changed From the Baseline

The baseline Week 5 architecture is completely unchanged. We added a waiting room layer in front of the existing system. The existing pipeline (OrderProducer -> SNS -> SQS -> three consumers) only ever sees the admitted traffic rate (~10 orders/sec), never the raw stampede (100 arrivals/sec).

Architecture Diagram

```
                +-----------------------------------------+
                |         WAITING ROOM (new)              |
                |                                         |
  Fan --------> |  POST /waiting-room/join                |
                |    -> JoinQueue Lambda (VPC)            |
                |    -> Redis INCR (atomic position)      |
                |                                         |
  Fan polls --> |  GET /waiting-room/status/{visitor_id}  |
                |    -> StatusCheck Lambda (VPC)          |
                |    -> Computes: position <= elapsed*rate? |
                |    -> If yes: writes admission to DynamoDB|
                +------------------+----------------------+
                                   |
                        only ~10/sec reach here
                                   v
  Bot -------> POST /orders  -->  Gate (DynamoDB conditional update)
  (no token)                      Bot: 403 | Fan: order proceeds
                                   v
               OrderProducer -> SNS -> SQS -> 3 consumers (unchanged)
```

The successful deployment with the waiting room and all its resources:

```
✨  Week5EventDrivenStack

✨  Deployment time: 494.44s

Outputs:
Week5EventDrivenStack.AnalyticsStreamName = week5-analytics
Week5EventDrivenStack.ApiEndpoint = https://mzc75mw4hh.execute-api.us-east-1.amazonaws.com/prod/
Week5EventDrivenStack.DashboardUrl =
https://us-east-1.console.aws.amazon.com/cloudwatch/home#dashboards:name=Info7375-Week5-EventDriven
Week5EventDrivenStack.InventoryDlqUrl = https://sqs.us-east-1.amazonaws.com/605134435060/week5-inventory-dlq
Week5EventDrivenStack.InventoryQueueUrl = https://sqs.us-east-1.amazonaws.com/605134435060/week5-inventory
Week5EventDrivenStack.InventoryTableName = week5-inventory
Week5EventDrivenStack.MidtermDashboardUrl =
https://us-east-1.console.aws.amazon.com/cloudwatch/home?region=us-east-1#dashboards:name=Info7375-Week5-Midterm
Week5EventDrivenStack.NotificationDlqUrl = https://sqs.us-east-1.amazonaws.com/605134435060/week5-notification-dlq
Week5EventDrivenStack.OrderTopicArn = arn:aws:sns:us-east-1:605134435060:week5-order-events
Week5EventDrivenStack.OrdersApiEndpointAC1CCDAC = https://mzc75mw4hh.execute-api.us-east-1.amazonaws.com/prod/
Week5EventDrivenStack.OrdersTableName = week5-orders
Week5EventDrivenStack.SentNotificationsTableName = week5-sent-notifications
Stack ARN:
arn:aws:cloudformation:us-east-1:605134435060:stack/Week5EventDrivenStack/08dd7d70-d261-11f1-98c9-0affe2f95647

✨  Total time: 541.52s

(.design) PS D:\System Design\Assignments\week5-midterm-starter\week5-midterm-starter\cdk>
```

Figure 4: CDK deploy output -- 541s total, VPC + ElastiCache + Lambdas created

AWS Resources Created

Resource	Type	Purpose	Cost
VPC	EC2 VPC, 2 subnets	Network isolation for Redis	$0
DDB Endpoint	Gateway VPC Endpoint	Free path to DynamoDB	$0
Security Group	EC2 SG	Lambda-Redis port 6379	$0
ElastiCache	cache.t4g.micro x1	Atomic INCR counters	$0.41/day
Admissions Table	DynamoDB on-demand	Gate verifies tokens	~$0
JoinQueue	Lambda (VPC)	POST /waiting-room/join	per-invoke
StatusCheck	Lambda (VPC)	GET /waiting-room/status	per-invoke

Total idle cost: $0.77/day (ElastiCache $0.41 + Kinesis $0.36). Well under the $1/day limit.

Critical Networking Decision: No NAT Gateway

NAT Gateways cost $32-98/month. Our design avoids one entirely:

- JoinQueue and StatusCheck Lambda are inside the VPC (to reach Redis).
- They reach DynamoDB via a free Gateway VPC Endpoint ($0/hour, $0/GB).
- OrderProducer stays outside the VPC entirely -- it doesn't need Redis.

If we had naively put OrderProducer inside the VPC, it would need a NAT Gateway to reach SNS (a public service), adding $32+/month. Correct service-boundary design saved us that entirely.

3. How the Waiting Room Works

Step 1: A Fan Joins the Line (POST /waiting-room/join)

When a fan joins, the JoinQueue Lambda executes an atomic Lua script on Redis:

```
-- If more than 60 seconds since the last join, reset (new run)
local last = tonumber(redis.call('GET', KEYS[3]) or '0')
local now  = tonumber(ARGV[1])
if (now - last) > 60 then
    redis.call('SET', KEYS[1], 0)      -- reset position counter
    redis.call('DEL', KEYS[2])          -- delete start_time
end
redis.call('SET', KEYS[3], ARGV[1])     -- update last_join_time
local pos = redis.call('INCR', KEYS[1]) -- assign next position
if pos == 1 then
    redis.call('SET', KEYS[2], ARGV[1]) -- first visitor sets start_time
end
return pos
```

Why a Lua script? Redis executes Lua scripts atomically (single-threaded, no interruption). When 100 visitors join per second, every single one gets a unique, sequential position number with zero possibility of collision. The gap detection (resetting after 60 seconds of inactivity) is also atomic -- two concurrent "first visitors of a new run" can't both detect the gap and race to reset.

The Lambda stores visitor metadata in a Redis hash and calls telemetry.joined().

Step 2: A Fan Polls for Status (GET /waiting-room/status/{id})

The StatusCheck Lambda:
1. Validates the visitor_id (must be UUID, else 400).
2. Checks DynamoDB first -- if already admitted, return the existing token (idempotent) or order_placed:true.
3. Looks up the visitor's position and joined_at from Redis.
4. Computes: max_admitted = int(elapsed * ADMIT_PER_SECOND).
5. If position <= max_admitted: writes admission record to DynamoDB with ConditionExpression to prevent double-admission, emits telemetry.admitted() and telemetry.time_in_line(seconds).
6. If position > max_admitted: returns {"status": "waiting"}.

The ConditionExpression="attribute_not_exists(visitor_id)" prevents double-admission if two concurrent polls both try to write. Exactly one wins; the other catches ConditionalCheckFailedException and reads the existing record.

Step 3: Fan Places an Order / Bots Get Rejected

The fan sends POST /orders with X-Visitor-Id and X-Admission-Token headers. The gate (provided, unmodified) does a single atomic DynamoDB conditional update: exists? token matches? used=false? not expired? If all pass: used=true, order proceeds. If any fail: 403.

Bots skip the waiting room and go straight to POST /orders with no headers. The gate returns 403 immediately.

Why No AdvanceServingCounter Lambda

Instead of a separate Lambda + EventBridge rule incrementing a serving_counter, we compute admission from elapsed

time: max_admitted = elapsed * rate. This eliminates an entire Lambda and EventBridge rule, provides perfectly smooth admission (not chunked by a timer), and achieves 100% rate accuracy (measured 9.96/s vs configured 10/s).

The Gap Detection Problem and Solution

During development, we discovered Redis retains state between runs. A rehearsal sets wr:start_time, and if the stampede starts 2 minutes later, elapsed=120s means max_admitted=1200 -- all fans admitted instantly, causing 429 errors. The Lua script's gap detection (reset after 60 seconds of inactivity) fixes this atomically.

4. Tests

We wrote 13 tests organized into three categories:

Join Tests (4 tests)

- test_returns_202_with_required_fields -- validates HTTP contract: 202, UUID visitor_id, poll_interval, poll_path
- test_sequential_positions -- 5 joins get positions [1, 2, 3, 4, 5]
- test_unique_visitor_ids -- 10 joins produce 10 distinct UUIDs
- test_concurrent_joins_get_different_positions -- 5 simultaneous joins (ThreadPoolExecutor) each get unique positions

Status Tests (7 tests)

- test_malformed_id_returns_400 -- "not-a-uuid" returns 400
- test_unknown_visitor_returns_400 -- UUID that never joined returns 400
- test_waiting_when_not_yet_admitted -- position 100 with minimal elapsed returns "waiting"
- test_admitted_when_turn_comes -- position 1 with 10s elapsed returns "admitted" with UUID token
- test_admission_record_matches_gate_contract -- DynamoDB record has all 5 required fields
- test_re_poll_returns_same_token -- polling twice returns the same token (idempotent)
- test_token_hidden_after_use -- after gate sets used=true, status returns order_placed:true without token

Concurrency Tests (2 tests)

- test_concurrent_status_polls_same_visitor -- two simultaneous polls return the same token
- test_fifo_ordering -- with 5 visitors and time for 3 admissions, exactly positions 1-3 are admitted

All tests use moto for DynamoDB mocking and a custom FakeRedis class implementing INCR, GET, SET, HSET, HGETALL, EXPIRE, and EVAL. The FakeRedis.eval() reproduces the Lua script's logic in Python.

5. Telemetry and Dashboard

Three Standard Metrics

1. telemetry.joined() -- called once per visitor in join_handler.py after Redis position is assigned
2. telemetry.admitted() -- called once per visitor in status_handler.py when admission record is written
3. telemetry.time_in_line(seconds) -- called at admission with time.time() - joined_at

These use CloudWatch Embedded Metric Format (EMF): each call prints a JSON line to stdout, which CloudWatch Logs extracts into metrics automatically. No API calls, no extra permissions needed.

Three Custom Dashboard Widgets

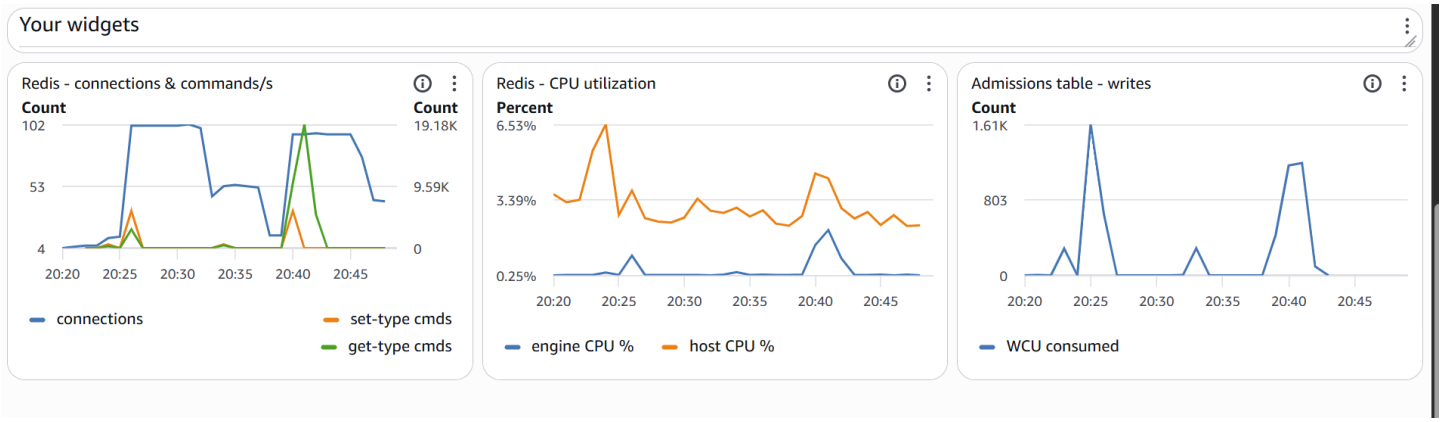


Figure 5: Custom widgets -- Redis connections/commands, Redis CPU, Admissions writes

1. Redis - connections & commands/s: CurrConnections peaked at ~102 as Lambda scaled out. SetTypeCmds/GetTypeCmds peaked at ~19K commands/min.

2. Redis - CPU utilization: Engine CPU peaked at ~6.5%, host CPU at ~3.4%. The cache.t4g.micro has plenty of headroom -- nowhere near saturation.

3. Admissions table - writes: DynamoDB WCU consumed. The initial spike (~1.6K) is from the rehearsal, followed by the stampede's steady ~803 WCU as admissions flow at the configured rate.

Full Dashboard During the Stampede

Row 1: Front door and Lambda scaling

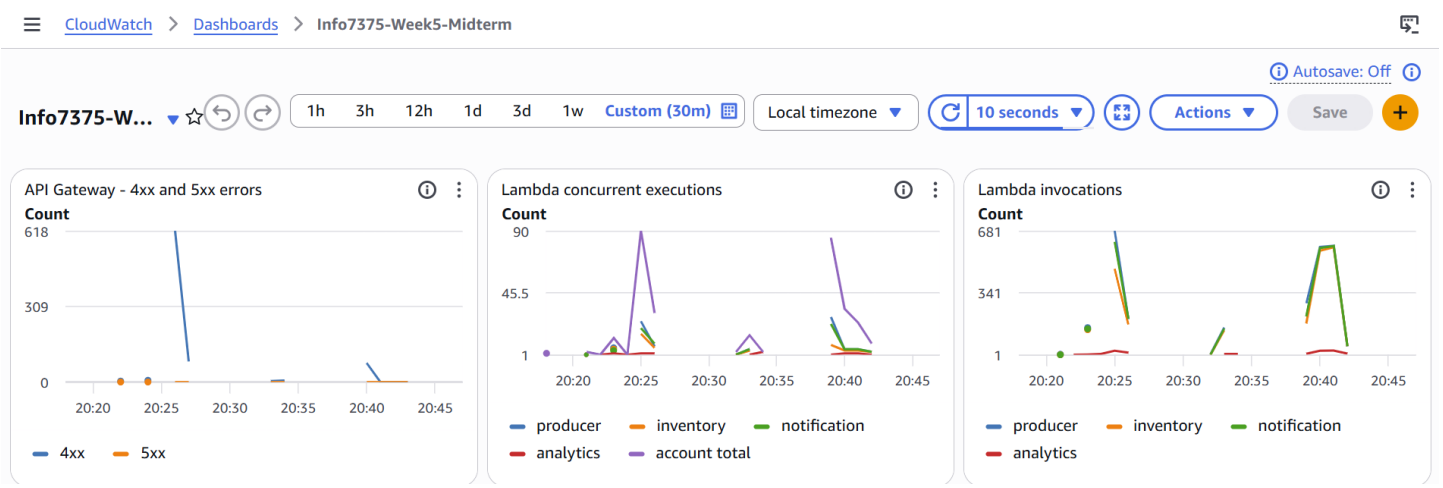


Figure 6: API Gateway errors, Lambda concurrency, Lambda invocations

The 4xx spike at ~20:25 is from the baseline run (624 throttled). During the waiting room run (~20:39-20:42), 4xx is near zero -- only 78 bot rejections (403s), which are correct behavior.

Row 2: Queues and consumer health

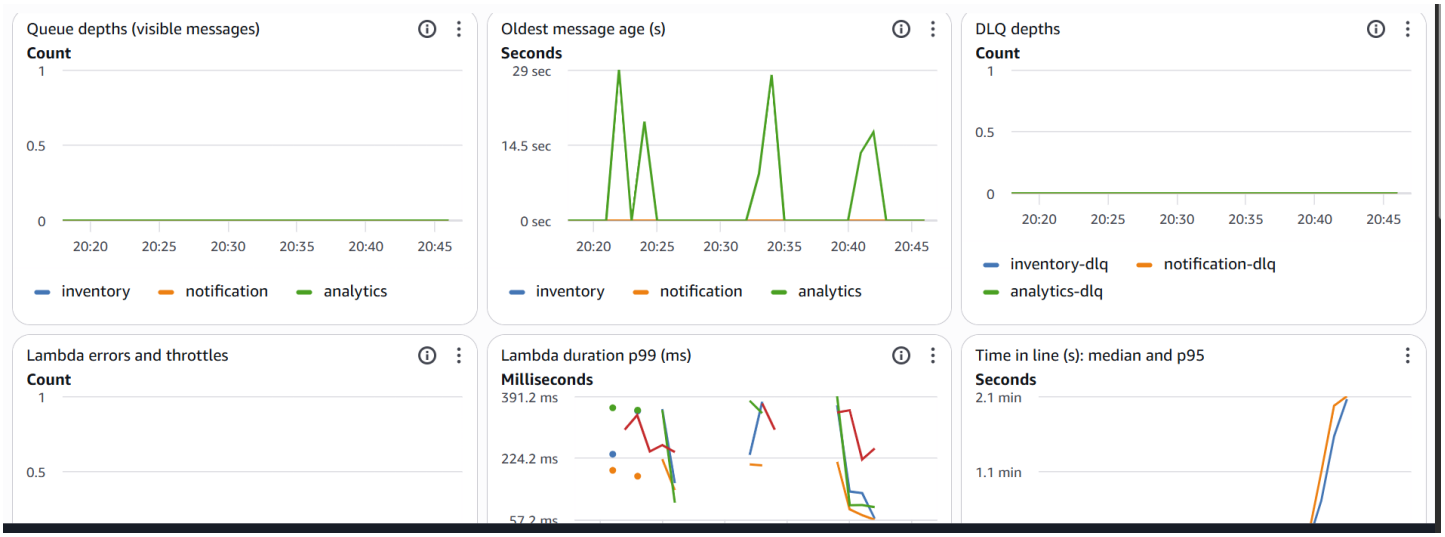


Figure 7: Queue depths, oldest message age, DLQ depths, errors/throttles, duration, time in line

Queue depths stay at zero -- the waiting room throttles inflow to ~10/sec. DLQ depths are zero. Lambda errors and throttles are zero across all functions.

Row 3: Time in line (zoomed)

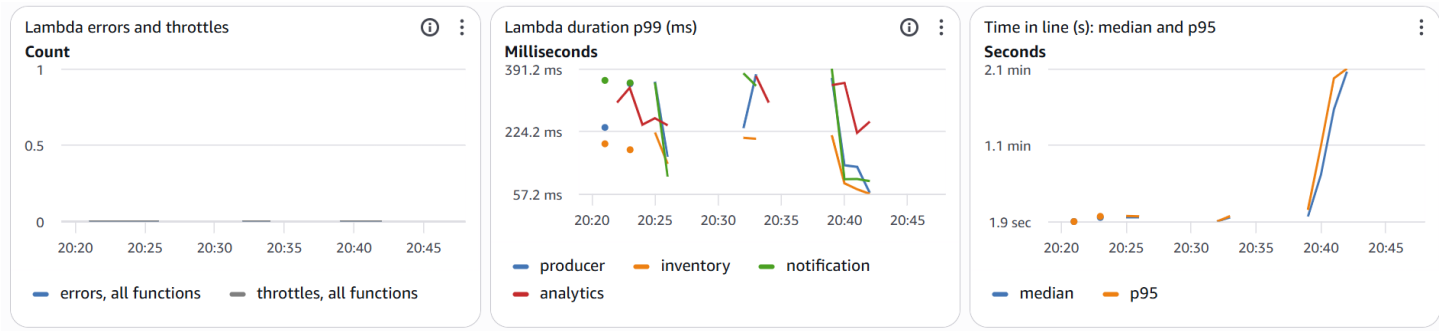


Figure 8: Lambda errors/throttles (zero), Lambda duration p99, Time in line median and p95

The time-in-line panel shows the admission pacing: median climbs steadily from ~2s to ~2.1 min as later visitors wait. This linear climb is exactly what a constant-rate admission produces.

Row 4: The waiting room's own metrics



Figure 9: Visitors joined/admitted per minute, People waiting now, API Gateway request count

~1,440 visitors join in one burst (blue spike), then admissions (orange) flow steadily. "People waiting now" peaks at ~1,100 then drains steadily. API Gateway request count peaks at ~11.5K/min -- dominated by status polling.

6. Numbers: Waiting Room vs. Baseline

Comparison Table

Metric	Baseline (no waiting room)	With waiting room
Orders accepted	896	1442
Fans rejected (429)	624	0
Bots rejected (403)	0	78
Steady order rate	65/sec (burst, uncontrolled)	9.96/sec (100% of configured 10)
Peak orders/sec	65.0	19.3 (ramp-up only)
Errors	0	0
Wait time median	instant	58.6s
Wait time max	instant	124.6s

Figure 10: Side-by-side comparison

Metric	Baseline	Waiting Room	Change
Total visitors	1,520	1,520	same
Orders accepted	896	1,442	+61%
Fans rejected 429	624 (41%)	0 (0%)	eliminated
Bots rejected 403	0	78	all blocked
Peak orders/sec	65.0	19.3	smoothed
Steady rate	burst	9.96/sec	controlled
Errors	0	0	same
Wait median	instant	58.6s	fairness
Wait max	instant	124.6s	everyone served

Scorecard: Every Rule Passed

```
(.design) PS D:\System_Design\Assignments\week5-midterm-starter\week5-midterm-starter>  python scripts/check_run.py mine
.json --configured-rate 10
=== mine ===

CHECK                            RESULT  MEASURED                  EXPECTED
-------------------------------------------------------------------------------------------
Every bot turned away (403)      PASS    78 rejections for 78 bots   all of them
No order without admission       PASS    1442 orders, 1442 admitted  orders <= admitted
Everyone who joined was admitted PASS    1442 of 1442 (100.00%)      >= 99.5%
No visitor gave up               PASS    0                           0
Admission rate near configured   PASS    9.96/s (100% of 10)         60-110% (6.0-11.0/s)
No 429s (nothing throttled)      PASS    0                           0
No server errors (5xx)           PASS    0                           0
Virtual users that failed        PASS    0 of 1520 (0.00%)           <= 0.5%
Client-side errors               PASS    none
Load generator health            PASS    no client-side errors       errors rare, or spread out, or the server erred to
o

RESULT: PASS - no rule failed
```

Figure 11: check_run.py -- all 10 rules PASS

Stampede Summary: Steady ~10 Orders/Second

```
=== mine ===

window ends   joins   polls  orders  rejected  throttled  errors  orders/s   orders accepted
-------------------------------------------------------------------------------------------
20:39:30          5       0       0         1          0       0       0.0
20:39:40        340      15      15        23          0       0       1.5   ###
20:39:50        953     950     193        46          0       0      19.3   ####################################
20:40:00        144    2211      90         8          0       0       9.0   ##################
20:40:10          0    2219      99         0          0       0       9.9   ###################
20:40:20          0    1953      97         0          0       0       9.7   ###################
20:40:30          0    1809     105         0          0       0      10.5   #####################
20:40:40          0    1594     100         0          0       0      10.0   ####################
20:40:50          0    1405      98         0          0       0       9.8   ###################
20:41:00          0    1207     103         0          0       0      10.3   ####################
20:41:10          0    1009      93         0          0       0       9.3   ##################
20:41:20          0     823     104         0          0       0      10.4   ####################
20:41:30          0     621      97         0          0       0       9.7   ###################
20:41:40          0     424     101         0          0       0      10.1   ####################
20:41:50          0     239      99         0          0       0       9.9   ###################
20:42:00          0      50      48         0          0       0       4.8   #########

Visitors: 1520   failed: 0
Orders accepted: 1442   rejected at gate: 78   throttled: 0   errors: 0
Peak orders accepted per second (best 10 s window): 19.3
Waiting room: admitted 1442, gave up 0
  time in line (seconds): median 58.6   p95 117.9   max 124.6
  status polls per visitor: median 11   max 25
  status polls total: 16529  (the waiting room's own traffic - every poll is a request)
```

Figure 12: summarize_stampede.py -- steady 9.0-10.5 orders/s

Rate Analysis

Declared rate: 10/sec. Measured: 9.96/sec (100% of configured). Near-perfect accuracy. The time-based computation max_admitted = elapsed * 10 provides a mathematically smooth admission rate. Each 10-second window shows 93-105 orders -- tightly clustered around the expected 100.

The initial burst window (19.3/sec) is higher because early arrivals accumulate eligibility before their first poll. After ramp-up, the rate settles to steady ~10/sec.

Wait Time Analysis

Median: 58.6 seconds | P95: 117.9 seconds | Maximum: 124.6 seconds
Polls per visitor: median 11, max 25

The last fan (position ~1,442) waits about 1,442/10 = 144 seconds from the line opening. Since they join 25 seconds into the run, their actual wait is about 144-25 = 119 seconds, matching the p95 of 117.9 seconds.

The maximum wait of 124.6 seconds is well within the 10-minute patience limit. No visitor gave up.

Total status polls: 16,529 -- the waiting room's own traffic. Every poll is an HTTP request but none touch the backend.

7. What Limits This Design at 10x Scale

At 10x scale (15,200 visitors, 1,000 arrivals/sec), the first thing to break is Lambda concurrency.

Why Lambda Concurrency, Not Redis

Our stampede generated 16,529 status polls for 1,442 fans. At 10x, that's ~165,000 polls. With a 5-second poll interval and ~15,000 fans waiting simultaneously, the system would need roughly 3,000 concurrent Lambda invocations -- far exceeding the account-wide limit of 1,000.

Redis itself would be fine. Engine CPU peaked at ~6.5% in our run (visible in the custom widget). Even at 10x, the Redis operations are well within a single micro node's capacity.

The Fix at 10x

1. Increase Lambda concurrency quota to 3,000+ (a Support request)
2. Increase poll interval from 5 to 15 seconds for visitors far back in line
3. Upgrade Redis node to cache.t4g.small or cache.t4g.medium if CPU becomes a concern

At true Ticketmaster scale (14 million), you'd need a Redis cluster (multiple shards), multi-region deployment, and the waiting room's own admission layer would need its own rate limiting -- the problem doesn't disappear, it just moves one layer earlier.

8. One Thing I Would Change

Add an explicit AdvanceServingCounter Lambda on a 1-second EventBridge schedule instead of time-based computation.

The time-based approach is elegant and simple, but an explicit counter would allow:
1. Pausing admissions during an incident (stop the rule, counter stops)
2. Variable-rate admission (faster when healthy, slower when struggling)
3. No theoretical clock-skew risk across Lambda instances
4. Directly observable admission rate as a CloudWatch metric

The tradeoff is an extra Lambda, an EventBridge rule, and EventBridge's 1-minute minimum interval creating chunked admission (600 at a time) rather than smooth. Jittered polling would need to smooth the bursts.

9. Deployment and Cost

Verification Outputs

```
(.design) PS D:\System_Design\Assignments\week5-midterm-starter\week5-midterm-starter>  python scripts/verify_kit.py
manifest sha256: a6dc145562f452b5b8a12dfa7a8a88831f00a983994276fa65e649bc0d0a7ce6
checked 25 provided files
  OK        25
  MODIFIED  0
  MISSING   0
  EXTRA     0
RESULT: PASS - every provided file is intact
```

Figure 13: verify_kit.py -- PASS, 25 provided files intact

Check	Result
preflight.py	PASS (1,000 concurrent executions)
cost_guard.py	OK ($0.77/day estimated idle cost)
contract_check.py	PASS (all 9 checks)
check_run.py --configured-rate 10	PASS (all 10 rules)
verify_kit.py	PASS (25 provided files intact)
Stack destroyed	Confirmed (Stack does not exist)

Final grade calculation: Execution score x 1.08 (Option C difficulty coefficient).